

Universidad Nacional Autónoma de México

Facultad de Ingeniería

Sistemas Operativos



Grupo 1

Ing. Angel Isidro Mercado

Tarea 7. Herramientas de trabajo

320304435 - Cabrera Rojas Oscar

Ciudad Universitaria, CDMX a 05 de noviembre del 2026

Índice

1. Fundamentos y arquitectura del escritorio remoto **1**
- 1.1. Definición y modelo cliente-servidor . . . 1
- 1.2. Evolución histórica: de terminales de texto a entornos gráficos . . . 2
- 1.3. Taxonomía técnica frente a tecnologías afines . . . 2

2. Protocolos de comunicación y subsistemas del sistema operativo **4**
- 2.1. Protocolos de transporte gráfico (RFB, RDP y WebRTC) . . . 4
  - 2.1.1. Protocolo de búfer de cuadro remoto (RFB) . . . 4
  - 2.1.2. Protocolo de escritorio remoto (RDP) . . . 4
  - 2.1.3. Comunicación web en tiempo real (WebRTC) . . . 5
- 2.2. Captura de pantalla e inyección de eventos en el sistema operativo . . . 5
- 2.3. Mecanismos de penetración de NAT y cortafuegos . . . 6
- 2.4. Seguridad, criptografía y autenticación en sesiones remotas . . . 7

3. Chrome Remote Desktop **7**
- 3.1. Arquitectura e infraestructura basada en WebRTC . . . 7
- 3.2. Procedimiento de configuración, vinculación y despliegue . . . 8
  - 3.2.1. Acceso remoto desatendido (Me2Me) . . . 8
  - 3.2.2. Asistencia remota bajo demanda (It2Me) . . . 9
- 3.3. Integración como servicio del sistema operativo y políticas de seguridad . . . 9

4. LogMeIn **9**
- 4.1. Arquitectura de intermediación en la nube y pasarelas de conexión . . . 10
- 4.2. Procedimiento de instalación, registro y administración de hosts . . . 10
- 4.3. Capacidades de gestión centralizada y modelo de cifrado . . . 11

5. TeamViewer **11**
- 5.1. Red de enrutamiento global y direccionamiento por identificador dinámico . . . 11
- 5.2. Procedimiento de instalación y modalidades de acceso . . . 12
- 5.3. Esquema criptográfico de sesión y herramientas de administración . . . 12

6. ShowMyPC **13**
- 6.1. Arquitectura basada en VNC y tunelización SSH . . . 13
- 6.2. Procedimiento operativo de generación de sesiones y asistencia . . . 14
- 6.3. Análisis técnico del modelo de conexión inversa . . . 15

7. Mobizen **15**
- 7.1. Arquitectura de réplica y control para dispositivos móviles . . . 15
- 7.2. Modos de conexión: enlace USB (ADB) y transporte inalámbrico . . . 16
- 7.3. Servicios del sistema operativo móvil, permisos de accesibilidad y captura . . . 17

8. Análisis comparativo de herramientas de escritorio remoto **17**
- 8.1. Matriz de capacidades técnicas, protocolos y rendimiento . . . 18
- 8.2. Evaluación de modelos de despliegue, seguridad y limitaciones de plataforma 18

9. Pruebas de funcionamiento práctico con TeamViewer **19**
9.1. Entorno experimental, topología de red y metodología de instrumentación . . 19
9.2. Opción 1: Control entre computadoras de escritorio y portátiles 20
9.3. Opción 2: Control interactivo con dispositivos móviles 21
9.3.1. Control de estación de escritorio desde dispositivo móvil (Móvil a PC) 21
9.3.2. Control remoto del dispositivo móvil desde estación de escritorio (PC a Móvil) 22

10. Conclusiones personales **23**

Referencias **24**

1. Fundamentos y arquitectura del escritorio remoto

El escritorio remoto permite desacoplar la ejecución lógica y el estado de los procesos respecto a los dispositivos periféricos encargados de la presentación visual y la captura de las entradas del usuario. Este principio garantiza que una sesión interactiva completa, administrada por el núcleo de un sistema operativo anfitrión, pueda ser visualizada y manipulada en tiempo real a través de una red de datos por un cliente remoto.

1.1. Definición y modelo cliente-servidor

Un escritorio remoto es un subsistema de software distribuido que proyecta el entorno gráfico de una computadora anfitriona hacia una computadora cliente mediante una red, manteniendo sincronización bidireccional continua. A diferencia de los servicios de compartición de archivos o ejecución por lotes, el escritorio remoto conserva el contexto de ejecución, el espacio de direcciones de memoria virtual y las estructuras del gestor de ventanas en el equipo anfitrión, transfiriendo únicamente la representación gráfica procesada y recibiendo las señales periféricas del usuario.

Como establecen Stallings (2018, §18.1, p. 827) y Carretero Pérez et al. (2001, §5.2, p. 232), esta arquitectura se fundamenta en el paradigma cliente-servidor para sistemas distribuidos mediante paso de mensajes:

- **Servidor o anfitrión (*host*):** Aloja la unidad central de procesamiento (CPU), la memoria física y el sistema operativo (SO) que ejecuta los procesos. Captura las modificaciones en el búfer de cuadro (*framebuffer*), las comprime y las transmite por red, al tiempo que recibe eventos de teclado y ratón para inyectarlos en la cola del SO como interrupciones sintéticas.
- **Cliente o visor (*viewer*):** Terminal operada por el usuario responsable de descomprimir y renderizar las actualizaciones gráficas, capturar los eventos periféricos locales y transmitirlos hacia el anfitrión.



Imagen 1: Arquitectura genérica cliente-servidor y desacoplamiento de capas funcionales (Stallings, 2018, §18.1, p. 828).

Dentro de la taxonomía de configuraciones cliente-servidor (Stallings, 2018, §18.1, p.

831), el escritorio remoto corresponde al modelo de procesamiento basado en servidor o cliente ligero (*thin client*), donde la lógica de aplicación y los datos residen en el anfitrión (Img. 1). La interacción opera en un bucle cerrado: la captura de entradas en el cliente genera paquetes transmitidos al host, donde se inyectan en las aplicaciones; estas actualizan la memoria gráfica y el host extrae, comprime y envía los cambios de vuelta al cliente. Para garantizar interactividad fluida, la latencia de ida y vuelta (*round-trip time*, RTT) debe mantenerse por debajo de los 100 ms, complementándose con canales virtuales multiplexados para audio, portapapeles y redirección de dispositivos periféricos.

1.2. Evolución histórica: de terminales de texto a entornos gráficos

El acceso remoto se originó en la década de 1960 con los sistemas de tiempo compartido (*time-sharing*) como CTSS y MULTICS en computadoras centrales (*mainframes*), diseñados para compartir ciclos de CPU entre múltiples operadores (Silberschatz et al., 2006, §1.2, p. 9). En los años setenta, los terminales tontos (*dumb terminals*) alfanuméricos como el DEC VT100 interactuaban mediante líneas serie asíncronas RS-232 y pseudoterminales (PTY) del sistema operativo. Posteriormente, la conectividad se estandarizó en redes TCP/IP con Telnet (Postel & Reynolds, 1983) y utilerías Berkeley (*rlogin*), sustituidas formalmente por el intérprete de órdenes seguro (SSH) debido a exigencias criptográficas (Ylonen & Lonvick, 2006).

La transición hacia interfaces gráficas sobre red surgió en 1984 en el MIT con el sistema de ventanas X (*X Window System* o X11) (Deitel et al., 2004; Scheifler & Gettys, 1986). En X11, el servidor gráfico corre en la estación local y las aplicaciones remotas actúan como clientes transmitiendo primitivas vectoriales de dibujo; no obstante, su naturaleza conversacional presentaba alta sensibilidad a la latencia sobre redes de área amplia (WAN).

A mediados de los noventa, Citrix introdujo MultiWin y el protocolo ICA sobre Windows NT 3.51 para habilitar sesiones gráficas multiusuario concurrentes, tecnología incorporada nativamente por Microsoft en Windows NT 4.0 Terminal Server Edition con el protocolo de escritorio remoto (RDP) (Microsoft Corporation, 2024). Paralelamente, AT&T Laboratories Cambridge desarrolló la computación virtual en red (VNC) basada en el protocolo de búfer de cuadro remoto (RFB), estandarizado en la RFC 6143 (Richardson & Levine, 2011; Richardson et al., 1998), que adoptó una premisa universal independiente de plataforma operando directamente sobre rectángulos de píxeles. Actualmente, estas arquitecturas convergen hacia la compresión de video por hardware (H.264, VP9, AV1) y la conectividad web interactiva con WebRTC.

1.3. Taxonomía técnica frente a tecnologías afines

Resulta indispensable distinguir formalmente el escritorio remoto frente a tres tecnologías afines que operan en capas disímiles del sistema operativo (Img. 2):

- **Escritorio remoto interactivo (RDP, RFB/VNC):** Extiende la sesión gráfica del sistema operativo con control bidireccional completo a nivel de búfer de cuadro y controladores de entrada.
- **Shell remota segura (SSH):** Canal cifrado que abstrae un flujo de caracteres vinculado a una PTY del kernel para interfaces de línea de comandos (Ylonen & Lonvick, 2006), con un consumo de ancho de banda y latencia mínimos.



Imagen 2: Clasificación de esquemas de procesamiento cliente-servidor y modelo de cliente ligero (Stallings, 2018, §18.1, p. 831).

- **Infraestructura de escritorios virtuales (VDI):** Provee máquinas virtuales aisladas sobre hipervisores nativos (*bare-metal*) en centros de datos, integrando capas de balanceo y orquestación masiva.
- **Duplicación de pantalla (*Screencasting* / *Mirroring*):** Transmisión unidireccional de video comprimido desde la GPU hacia pantallas receptoras pasivas (Miracast, Cast, AirPlay), priorizando sincronía audiovisual sin canal de retorno de control.

La Tabla 1 sintetiza las diferencias operativas entre estos enfoques.

Tabla 1: Taxonomía comparativa de tecnologías de acceso y control remoto.

Tecnología	Capa de abstracción en el SO	Interacción y canal de retorno	Ancho de banda típico	Protocolos representativos
Escritorio Remoto	Búfer de cuadro, gestor de ventanas, API gráfica	Bidireccional completa (eventos periféricos y audio)	1 a 15 Mbps (adaptativo a la actividad)	RDP, RFB (VNC), WebRTC, Citrix ICA
Shell Segura (SSH)	Pseudoterminal (PTY), flujo de caracteres del kernel	Bidireccional basada en texto (I/O estándar)	$<$ 100 kbps (extremadamente ligero)	SSHv2 (RFC 4251), Telnet (RFC 854)
VDI	Hipervisor de tipo 1, máquina virtual aislada	Bidireccional completa con orquestación de recursos	2 a 20 Mbps por sesión concurrente	PCoIP, Blast Extreme, HDX, RDP
Duplicación de Pantalla	Salida del codificador de video de la GPU	Unidireccional (emisión pasiva de video y audio)	5 a 30 Mbps (flujo continuo de video)	Miracast (Wi-Fi Display), Cast, AirPlay

Esta distinción teórica y operativa permite situar con precisión el alcance técnico de las herramientas de escritorio remoto analizadas a lo largo del presente trabajo, las cuales

operan en el plano de la interacción bidireccional con interfaces gráficas, equilibrando la fidelidad visual frente a las restricciones de ancho de banda y latencia de las redes de datos.

2. Protocolos de comunicación y subsistemas del sistema operativo

La viabilidad de una sesión remota depende de la interacción armónica entre la pila de protocolos de red y los subsistemas internos del sistema operativo. Mientras el transporte en red garantiza la entrega oportuna de fotogramas y comandos, el núcleo del sistema anfitrión debe interceptar las superficies del adaptador gráfico e inyectar eventos periféricos sintéticos sin degradar el rendimiento global.

2.1. Protocolos de transporte gráfico (RFB, RDP y WebRTC)

El transporte gráfico en red concilia un ancho de banda considerable con latencias reducidas para preservar la ilusión de interactividad local. Destacan tres arquitecturas representativas:

2.1.1. Protocolo de búfer de cuadro remoto (RFB)

Formalizado en la RFC 6143 por Richardson y Levine (2011) y concebido por Richardson et al. (1998), RFB opera sobre una abstracción de píxeles homogénea e independiente del sistema de ventanas. Su principio medular es el control de flujo impulsado por el cliente (*client-driven update protocol*): el servidor no emite fotogramas de forma espontánea, sino que responde a requerimientos explícitos (`FramebufferUpdateRequest`) enviando únicamente las regiones rectangulares modificadas (`FramebufferUpdate`). Este diseño previene la saturación de búferes en clientes lentos o enlaces congestionados. Para optimizar el ancho de banda, RFB integra codificaciones adaptativas que abarcan desde el mapeo sin compresión (*Raw*) y el agrupamiento por longitudes de corrida (*RRE*, *Hextile*, *ZRLE*) hasta la compresión de gradientes con cuantificación JPEG en el esquema *Tight*.

2.1.2. Protocolo de escritorio remoto (RDP)

Desarrollado por Microsoft y documentado en Microsoft Corporation (2024), RDP adopta una arquitectura modular de alto nivel derivada de las normas ITU-T T.120. Su diseño multiplexa canales virtuales estáticos (SVC) y dinámicos (DVC) para enrutar de forma aislada flujos de audio, portapapeles, dispositivos serie, tarjetas inteligentes y discos locales. Históricamente, RDP transmitía primitivas vectoriales de dibujo de la interfaz GDI de Windows, reduciendo el consumo de red al ejecutar el renderizado en el cliente. Ante interfaces compuestas modernas con aceleración 3D, el protocolo evolucionó incorporando la tecnología RemoteFX y canalizaciones basadas en AVC/H.264 (perfiles 4:2:0 y 4:4:4), delegando la codificación y decodificación de fotogramas a los motores de la GPU.

2.1.3. Comunicación web en tiempo real (WebRTC)

Estandarizado en la RFC 8825 por Alvestrand (2021), WebRTC fundamenta su transporte en el protocolo de datagramas de usuario (UDP) para priorizar la inmediatez sobre la retransmisión estricta de pérdidas no críticas, desacoplando su operación en dos planos:

- **Plano audiovisual:** La superficie del escritorio se codifica en video (VP8, VP9, H.264 o AV1) y fluye mediante el protocolo de transporte en tiempo real seguro (SRTP), incorporando algoritmos de control de congestión adaptativo (GCC) que ajustan el *bitrate* y la resolución según la fluctuación de retardo (*jitter*) y las pérdidas de red.
- **Plano de datos:** Los eventos de teclado y ratón se canalizan a través de canales de datos SCTP encapsulados sobre DTLS/UDP, habilitando entrega inmediata sin retransmisiones redundantes y eliminando el bloqueo de cabeza de línea (*head-of-line blocking*) propio de TCP.

2.2. Captura de pantalla e inyección de eventos en el sistema operativo

El agente anfitrión debe interactuar con el espacio del núcleo y los controladores de dispositivos para extraer fotogramas del adaptador de video y emular interrupciones físicas de entrada.



Imagen 3: Intercepción de superficies en el subsistema gráfico e inyección de eventos sintéticos en el núcleo del sistema operativo.

Como esquematiza la Img. 3, el software enlaza la pila de red con el espacio de memoria del sistema operativo:

- **Captura de pantalla asistida por hardware:** Frente a las técnicas históricas de raspado pasivo por sondeo (`BitBlt` en GDI o `XGetImage` en X11), que saturaban la CPU y el bus PCIe transfiriendo memoria desde la VRAM hacia la RAM, los sistemas modernos operan sobre composición acelerada. En Microsoft Windows, Desktop Duplication API (DDA) sobre DXGI y Direct3D 11 accede directamente a las texturas en

VRAM (`ID3D11Texture2D`) y recupera metadatos de regiones alteradas (*dirty rectangles*), permitiendo a la GPU codificar el flujo de video sin copias intermedias. En Linux, se transitó de la extensión `XDamage` en X11 al compositor Wayland mediante `xdg-desktop-portal` y PipeWire, donde se negocian descriptores de memoria compartida DMA-BUF de copia cero a nivel de kernel.

- **Inyección de eventos sintéticos:** En Windows, la inyección se ejecuta mediante `SendInput()` en `user32.dll`, que traduce las coordenadas absolutas y teclas virtuales a códigos de rastreo (*scan codes*) entregados a la cola global de `win32k.sys`. Para interactuar con cuadros de diálogo protegidos (UAC), el agente solventa el aislamiento de la Sesión 0 ejecutando componentes auxiliares en la sesión interactiva del usuario. En Linux, el demonio interactúa con el dispositivo virtual `/dev/uinput` gestionado por `evdev`, escribiendo estructuras `struct input_event` que el núcleo procesa idénticamente a las interrupciones de un periférico físico.

2.3. Mecanismos de penetración de NAT y cortafuegos

La mayoría de los nodos terminales operan confinados en redes privadas bajo traducción de direcciones de red (NAT, RFC 3022 de Srisuresh y Egevang (2001)) y cortafuegos con inspección de estado que descartan cualquier paquete entrante no solicitado. Para sortear esta restricción sin reenvío manual de puertos se recurre a dos esquemas:



Imagen 4: Recolección de candidatos ICE y penetración de NAT mediante STUN/TURN y pasarelas de retransmisión (Rosenberg et al., 2018, §2, p. 9).

- **Conectividad interactiva (ICE) con STUN y TURN:** Siguiendo la RFC 8445 (Rosenberg et al., 2018), ICE recolecta candidatos de host, reflexivos y de retransmisión (Img. 4). STUN (RFC 8489 de Petit-Huguenin et al. (2020)) permite a los nodos descubrir su dirección IP y puerto públicos reflexivos mediante transacciones de enlace, facilitando la perforación de puertos (*hole punching*) para establecer enlaces directos punto a punto (P2P). Si existen enrutadores con NAT simétrico o políticas estrictas que bloquean UDP, interviene TURN (RFC 8656 de Reddy et al. (2020)), actuando como nodo de retransmisión en la nube que conmuta el tráfico bidireccional.

- **Intermediación centralizada y túneles inversos:** Soluciones corporativas y herramientas de soporte implementan un modelo de intermediario (*broker*): el host establece y preserva una conexión TCP saliente permanente cifrada vía TLS hacia la nube sobre el puerto estándar 443 (HTTPS), universalmente habilitado en cortafuegos. Al registrarse con un identificador dinámico, los servidores de coordinación facilitan la negociación P2P o conmutan la sesión a través de un túnel inverso (*reverse tunnel*) gestionado en la nube.

2.4. Seguridad, criptografía y autenticación en sesiones remotas

El control remoto entrega facultades operativas totales sobre el equipo anfitrión, exponiéndolo a intercepción pasiva (*eavesdropping*), ataques de intermediario (MITM), fuerza bruta sobre puertos abiertos y secuestro de sesión (Stallings, 2018, §15.1, p. 658). Para mitigar estos vectores de riesgo, las arquitecturas seguras integran defensas multicapa:

1. **Intercambio de claves y autenticación mutua:** Acuerdos asimétricos basados en RSA (2048 o 4096 bits) o curvas elípticas (ECDH sobre Curve25519) con secreto perfecto hacia adelante (*forward secrecy*) para impedir la suplantación de identidad y el descifrado retroactivo.
2. **Cifrado simétrico autenticado:** Cifrado integral de fotogramas, comandos y canales auxiliares mediante AES-256-GCM o ChaCha20-Poly1305 (cifrados AEAD), garantizando confidencialidad e integridad criptográfica simultáneas frente a alteraciones de paquetes.
3. **Autenticación a nivel de red (NLA):** En protocolos como RDP, NLA valida las credenciales del usuario vía CredSSP o Kerberos antes de inicializar el subsistema gráfico o instanciar la pantalla de bienvenida, bloqueando intentos de denegación de servicio (DoS) contra el servidor gráfico.
4. **Contraseñas efímeras y autenticación multifactor:** Aplicando el principio de contraseñas de un único acceso expuesto por Carretero Pérez et al. (2001, §9.7, p. 531), las herramientas generan credenciales dinámicas de alta entropía que expiran al cierre de la sesión, complementadas con autenticación multifactor (MFA/2FA) y consentimiento manual del operador local.

3. Chrome Remote Desktop

Desarrollado dentro del árbol de código abierto de Chromium bajo el proyecto *Chromoting*, Chrome Remote Desktop converge estándares web y subsistemas nativos. Su innovación radica en prescindir de clientes propietarios pesados, transformando navegadores modernos (mediante HTML5, WebRTC, WebAssembly y aceleración gráfica) en visores de alto rendimiento y latencia reducida.

3.1. Arquitectura e infraestructura basada en WebRTC

El sistema desacopla la señalización en la nube del transporte directo de datos (The Chromium Projects, 2024), adoptando la pila WebRTC (RFC 8825) junto con servicios de directorio de Google.

Como ilustra la Img. 5, el software anfitrión implementa una arquitectura multiproceso bajo el principio de mínimo privilegio:



Imagen 5: Arquitectura multiproceso del host Chromoting y separación de privilegios mediante IPC Mojo (The Chromium Projects, 2024).

1. **Proceso demonio del host:** Ejecutado como proceso raíz (`remoting_me2me_host`) con privilegios del sistema (`LocalSystem` o *root*) para administrar el ciclo de vida de la sesión, la autenticación y la instanciación de los procesos de usuario.
2. **Proceso de sesión de escritorio:** Instancia (`remoting_desktop`) confinada al contexto del usuario interactivo, capturando el adaptador gráfico (DDA en Windows o PipeWire en Linux) e inyectando eventos periféricos.
3. **Proceso de conexión de pares:** Módulo (`remoting_peer_connection_process`) que aísla la pila WebRTC, negociación DTLS y codificación de video dentro de un entorno restringido (*sandbox*) sin acceso directo al sistema de archivos.
4. **Mojo IPC:** Canalización de comunicación interprocesos fuertemente tipada de Chromium que transfiere descriptores y memoria compartida con latencia nula.

La conexión opera en dos fases: señalización vía HTTPS/gRPC contra la infraestructura de Google para intercambiar descriptores SDP y credenciales efímeras; y plano de datos WebRTC, donde ICE negocia enlaces directos P2P mediante STUN (`stun.l.google.com:19302`) o conmuta a TURN (puerto 443). El video se comprime en VP8, VP9 o H.264 acelerado por GPU, y las entradas fluyen por canales SCTP/DTLS.

3.2. Procedimiento de configuración, vinculación y despliegue

Conforme a Google Help (2024), la herramienta ofrece dos modalidades operativas:

3.2.1. Acceso remoto desatendido (Me2Me)

Orientado al acceso permanente del propietario hacia su propio equipo:

1. **Aprovisionamiento del host:** Instalación del paquete nativo (`.msi`, `.deb` o `.pkg`) desde `remotedesktop.google.com/access`.
2. **Vinculación de cuenta:** Autorización mediante OAuth 2.0 enlazada a la cuenta de Google, registrando el identificador de la máquina en el directorio.

3. **Esquema criptográfico SPAKE2:** El usuario define un PIN local ($\geqslant 6$ dígitos). El PIN no se envía a Google ni se almacena en texto claro: cliente y anfitrión ejecutan el protocolo de intercambio de claves autenticado por contraseña SPAKE2 (*Simple Password-Based Encrypted Key Exchange*), derivando claves simétricas compartidas mediante un desafío criptográfico de conocimiento cero.

3.2.2. Asistencia remota bajo demanda (It2Me)

Diseñado para soporte técnico puntual entre usuarios:

1. **Generación de código efímero:** El host emite un código numérico aleatorio de 12 dígitos con vigencia estricta de 5 minutos desde `remotedesktop.google.com/support`.
2. **Consentimiento explícito e indicadores:** El operador ingresa el código, pero la sesión solo se activa tras la confirmación modal del usuario local ("Compartir"). Una barra persistente inferior notifica la conexión activa, permite la interrupción inmediata y emite recordatorios cada 30 minutos.

3.3. Integración como servicio del sistema operativo y políticas de seguridad

La adaptación al sistema operativo anfitrión difiere según la plataforma:

- **Windows:** Servicio del sistema (`remoting_host.exe`) bajo `LocalSystem`, facultado para interactuar antes del inicio de sesión y simular la secuencia segura (`Ctrl+Alt+Supr`), acoplándose al compositor DWM mediante DirectX DDA.
- **GNU/Linux:** Servicio administrado por `systemd` por usuario. Inicializa un servidor virtual independiente sin monitor (*headless*) empleando `Xvfb`, configurado mediante el archivo `~/.chrome-remote-desktop-session` y aprovisionable vía CLI sobre SSH.
- **macOS:** Demonio bajo `launchd` que requiere concesión explícita de permisos de "Grabación de pantalla" (ScreenCaptureKit) y "Accesibilidad" (Core Graphics).
- **Modo cortina y directivas empresariales:** Permite ocultar la pantalla física local e inhabilitar periféricos locales durante la conexión. Mediante GPO o directivas corporativas (Google Chrome Enterprise, 2024), se administran reglas como el control de penetración de cortafuegos (`RemoteAccessHostFirewallTraversal`), listas blancas de dominios corporativos (`RemoteAccessHostDomainList`) y la desactivación de asistencia bajo demanda.

4. LogMeIn

Fundada en 2003 como 3am Labs por Márton Anka y consolidada comercialmente como LogMeIn (actualmente GoTo), la plataforma transformó el acceso corporativo al sustituir la configuración manual de puertos e IP públicas por una arquitectura de intermediación en la nube soportada enteramente en protocolos web estándar.

4.1. Arquitectura de intermediación en la nube y pasarelas de conexión

El diseño de LogMeIn se basa en un esquema distribuido de tres entidades (Anka, 2007): el cliente o visor (*LogMeIn Client*), el anfitrión (*LogMeIn Host*) ejecutado como servicio del sistema operativo, y el clúster de pasarelas (*Gateway Cluster*) en la nube.



Imagen 6: Arquitectura de pasarela de conexión y túnel saliente de tres entidades en LogMeIn (Anka, 2007, §2, p. 4).

Como ilustra la Img. 6, la comunicación sortea cortafuegos mediante túneles salientes persistentes:

- **Túnel saliente y sincronización:** Al iniciar el anfitrión, el servicio abre una conexión TCP/TLS saliente hacia las pasarelas sobre el puerto 443 (HTTPS), universalmente admitido por cortafuegos con inspección de estado (Shockey, 2008). El enlace se mantiene activo mediante paquetes de latido (*heartbeat*) que impiden la expiración de tablas NAT.
- **Intermediación y retransmisión:** El visor solicita la sesión al portal web, notificando al anfitrión a través del conector abierto. La pasarela intenta coordinar una conexión P2P directa vía perforación de puertos; si las políticas de red lo impiden, conmuta a modo retransmisión (*relay*), enrutando los paquetes cifrados sin capacidad de descifrarlos.

4.2. Procedimiento de instalación, registro y administración de hosts

El despliegue combina instaladores interactivos individuales (`.msi` o `.pkg`) con distribución corporativa masiva mediante paquetes MSI personalizados con tokens criptográficos empotrados (`DeployToken`) desde LogMeIn Central, desplegables silenciosamente vía GPO o MECM (`/quiet`).

El acceso implementa un modelo de autenticación en dos niveles (*Dual Authentication Tier*):

1. **Nivel de servicio en la nube:** Autenticación en la consola web con credenciales maestras y factor múltiple obligatorio (MFA/TOTP).
2. **Nivel del sistema operativo local:** Al establecerse el canal, el agente exige ingresar las credenciales del sistema anfitrión (locales SAM o dominio de Active Directory). El demonio valida las credenciales contra la interfaz SSPI mediante `LogonUser()` en `advapi32.dll` y ejecuta suplantación de identidad (*impersonation*), subordinando la sesión a las listas de control de acceso (ACL) y privilegios locales.

4.3. Capacidades de gestión centralizada y modelo de cifrado

LogMeIn trasciende el control visual al integrar herramientas de gestión en segundo plano (*Background Management*) que operan sin invadir la pantalla local ni interrumpir al usuario físico: terminal interactiva remota (CMD o PowerShell con privilegios del sistema), administrador de tareas, monitor de servicios y eventos, editor del registro (`regedit`) y explorador de archivos.

Para flotas corporativas, incorpora el módulo One2Many para distribución masiva de scripts de mantenimiento, Wake-on-LAN distribuido (utilizando otros hosts encendidos de la misma subred como pasarelas de *magic packets*) y reinicio remoto persistente en Modo Seguro con funciones de red.

Conforme a las especificaciones de GoTo (GoTo Technologies USA, Inc., 2024), las sesiones se protegen mediante TLS 1.2/1.3 con certificados X.509 de 2048 bits y cifrado simétrico AES-256 (CBC/GCM) de extremo a extremo. El sistema genera bitácoras forenses en la nube e inyecta identificadores de evento en el registro de aplicaciones de Windows (*Application Event Log*), cumpliendo normativas SOC 2 Tipo II, ISO/IEC 27001, HIPAA y RGPD.

5. TeamViewer

Creada en 2005 en Göppingen, Alemania, por TeamViewer AG, la plataforma simplificó el acceso remoto sustituyendo direcciones IP y configuraciones complejas de enrutamiento por un identificador unívoco de máquina respaldado por una red global de conmutación.

5.1. Red de enrutamiento global y direccionamiento por identificador dinámico

La arquitectura opera sobre una red jerárquica de dos niveles (TeamViewer Germany GmbH, 2024b): clústeres maestros (*Master Cluster*), que resuelven identidades, estados de presencia y firmas públicas; y servidores de enrutamiento (*Routing Servers*), distribuidos globalmente para conmutar flujos de sesión.

Como ilustra la Img. 7, la conectividad se sustenta en:

- **TeamViewer ID:** Identificador permanente de 9 o 10 dígitos derivado de las características físicas de la máquina (dirección MAC, UUID de la placa base y número de serie del disco), preservando la identidad del equipo ante cambios de red o proveedor.
- **Transporte adaptativo y conmutación:** Utiliza preferentemente su protocolo binario en puertos TCP/UDP 5938. Si existen cortafuegos restrictivos, conmuta a TCP 443



Imagen 7: Topología de la red de TeamViewer: Servidores maestros, routers y conmutación P2P/Relay (TeamViewer Germany GmbH, 2024b, §3, p. 7).

(HTTPS) o TCP 80 (HTTP). Al iniciar una sesión, los extremos intentan enlazar directamente vía P2P mediante perforación de puertos UDP/TCP; si el entorno impone NAT simétrico, el tráfico se redirige a través del servidor de enrutamiento más próximo sin descifrar los paquetes.

5.2. Procedimiento de instalación y modalidades de acceso

La plataforma estructura su software en módulos especializados (TeamViewer Germany GmbH, 2024c): el cliente completo (*Full Client*) para control y administración; el módulo Host, instalado como servicio en segundo plano para acceso desatendido continuo a servidores; *QuickSupport* (QS), ejecutable sin instalación (*zero-install*) que genera credenciales temporales para soporte puntual; y *QuickJoin* para conferencias.

Para acceso autónomo sin confirmación presencial se configura “Acceso Fácil” (*Easy Access*), asociando el host a la cuenta de usuario mediante certificados digitales protegidos por 2FA, o mediante contraseñas estáticas de sistema complementadas con autenticación nativa de Windows.

5.3. Esquema criptográfico de sesión y herramientas de administración

La seguridad técnica se fundamenta en un modelo de cifrado de extremo a extremo (TeamViewer Germany GmbH, 2024b):

- **Criptografía híbrida:** Intercambio asimétrico de claves RSA de 4096 bits (almacenadas localmente en el almacén de seguridad del sistema operativo y nunca compartidas con los servidores de enrutamiento) con acuerdo Diffie-Hellman respaldado por secreto perfecto hacia adelante (PFS). Los flujos de video, comandos periféricos y archivos se cifran simétricamente con AES-256, impidiendo la inspección del tráfico en los nodos de retransmisión.

- **Defensa contra fuerza bruta:** Contraseñas dinámicas efímeras y un algoritmo de retardo exponencial ante intentos fallidos (*exponential backoff delay*: 1 s, 2 s, 4 s, hasta varias horas de penalización), inutilizando ataques automatizados.
- **Control de acceso y herramientas de sesión:** Incorpora listas de admisión y bloqueo (*Allowlist/Blocklist*), verificación de dispositivos de confianza (*Trusted Devices*) vía correo electrónico y 2FA, pantalla negra con inhabilitación de periféricos locales en el anfitrión, bloqueo automático del sistema operativo y grabación obligatoria de sesiones en formato cifrado (`.tvs`) para auditoría forense.

6. ShowMyPC

Desarrollada a partir de 2006 por ShowMyPC LLC, esta plataforma se consolidó como una de las primeras soluciones de soporte remoto ágiles basadas en tecnologías de código abierto. Frente a plataformas comerciales que desarrollaban pilas de protocolos propietarias e infraestructuras de directorio centralizadas, ShowMyPC adoptó un enfoque pragmático de integración: empaquetó el motor de renderizado gráfico de Virtual Network Computing (VNC) con la robustez criptográfica de Secure Shell (SSH), articulando un modelo de conexión asistida sin requerir privilegios de administrador ni alterar la configuración de red perimetral (ShowMyPC LLC, 2024).

6.1. Arquitectura basada en VNC y tunelización SSH

El diseño arquitectónico de ShowMyPC desacopla el procesamiento gráfico de la seguridad en el transporte mediante la convergencia de dos proyectos de código abierto ampliamente probados:

1. **Motor gráfico TightVNC:** Utiliza una variante optimizada de TightVNC (Kaplinsky, 2001), la cual implementa el protocolo de búfer de cuadro remoto (RFB, RFC 6143 de Richardson y Levine (2011)). El servidor VNC anfitrión se ejecuta de manera local y se vincula estrictamente a la interfaz de bucle invertido (*loopback*, dirección IP `127.0.0.1`). En esta configuración, el servidor rechaza cualquier intento de conexión directa proveniente de interfaces de red externas, previniendo accesos indebidos en la red de área local.
2. **Cliente de tunelización SSH:** Dado que el protocolo RFB original carece de mecanismos nativos de cifrado para el flujo de píxeles y eventos periféricos, ShowMyPC integra un cliente SSH embebido (derivado de OpenSSH y Plink). Este componente encapsula la totalidad del tráfico de la sesión dentro de un túnel cifrado bajo el estándar SSHv2 (Ylonen & Lonvick, 2006), empleando algoritmos simétricos robustos como AES y autenticación de integridad basada en HMAC.
3. **Pasarela pública de retransmisión (*stitching server*):** Entre el equipo anfitrión y el visor remoto interviene un clúster de servidores intermedios con direcciones IP públicas estáticas. La pasarela opera demonios SSH configurados para recibir solicitudes de reenvío de puertos remotos y actuar como conmutador de flujos de datos.

Como se esquematiza en la Img. 8, la arquitectura no expone ningún puerto de escucha del anfitrión hacia la Internet pública; el servidor intermedio en la nube orquesta la unión lógica de los canales cifrados establecidos por ambos extremos.



Imagen 8: Encapsulación de RFB en túnel inverso SSH y pasarela de retransmisión en ShowMyPC (ShowMyPC LLC, 2024).

6.2. Procedimiento operativo de generación de sesiones y asistencia

El ciclo operativo de ShowMyPC prioriza la reducción de la fricción operativa mediante un empaquetado monolítico autocontenido (*standalone wrapper*) que no exige instalación formal en el disco ni modificaciones en el registro del sistema operativo:

1. **Inicio de la compartición de pantalla (*Share Screen*):** El usuario que solicita asistencia ejecuta el binario y pulsa la opción de compartir. En ese instante, el contenedor extrae en memoria o en un directorio temporal de usuario las instancias mínimas de TightVNC Server y el cliente SSH.
2. **Apertura del túnel inverso y generación de clave:** El cliente SSH inicializa una conexión saliente autenticada hacia la pasarela de ShowMyPC, instruyendo al servidor intermedio mediante una directiva equivalente a `ssh -R` para enlazar un puerto efímero del servidor con el puerto local de TightVNC (`127.0.0.1:5900`). La pasarela devuelve un código de sesión dinámico estructurado en cuatro segmentos numéricos o alfanuméricos (por ejemplo, una clave de 12 dígitos).
3. **Unión del visor remoto (*View PC*):** El operador técnico introduce el código de sesión en la aplicación cliente o accede a través del visor web basado en HTML5. La pasarela valida la clave, identifica el túnel inverso abierto por el anfitrión y conmuta el flujo bidireccional, permitiendo al visor interactuar con el escritorio anfitrión con control total de teclado y ratón.
4. **Modalidad de acceso desatendido y RDP:** En entornos donde se requiere acceso permanente sin presencia de operador local, ShowMyPC provee un servicio anfitrión registrable con credenciales fijas. Asimismo, la herramienta soporta la tunelización del protocolo de escritorio remoto (RDP) de Windows a través del canal SSH, permitiendo aprovechar la aceleración gráfica nativa y compresión avanzada de Windows frente al renderizado basado en mapas de bits de VNC.

6.3. Análisis técnico del modelo de conexión inversa

La relevancia de ShowMyPC en la evolución de las herramientas de asistencia técnica radica en su resolución del problema de conectividad detrás de enrutadores NAT y cortafuegos perimetrales:

- **Superación de las limitaciones del VNC tradicional:** El despliegue estándar de VNC impone una arquitectura pasiva en el anfitrión: el servidor escucha peticiones entrantes en el puerto TCP 5900. En una red local moderna protegida por NAT (RFC 3022 de Srisuresh y Egevang (2001)), un cliente externo no puede alcanzar dicha dirección privada a menos que el administrador configure reglas manuales de reenvío de puertos (*port forwarding*) en el enrutador de borde y disponga de una IP pública enrutable.
- **Mecanismo de tunelización inversa:** ShowMyPC invierte la dirección de la llamada de red. Dado que los cortafuegos con inspección de estado bloquean el tráfico entrante pero autorizan por defecto las conexiones TCP salientes iniciadas desde la red interna (típicamente sobre los puertos estándar 22 de SSH, 443 de HTTPS o 80 de HTTP), el anfitrión inicia activamente la sesión hacia la pasarela. La tabla de estado del cortafuegos registra la conexión saliente y permite que los paquetes de retorno fluyan dentro del mismo canal lógicamente establecido, transportando la sesión remota sin requerir privilegios administrativos en el enrutador.
- **Compensaciones de diseño (*trade-offs*):** Si bien este modelo garantiza una compatibilidad excepcional en sistemas antiguos o entornos restringidos, introduce penalizaciones operativas frente a soluciones contemporáneas como WebRTC o TeamViewer. Al depender exclusivamente de TCP para transportar tanto el flujo de pantalla como las entradas periféricas dentro de un túnel SSH, la sesión es vulnerable al bloqueo de cabeza de línea ante pérdidas de paquetes. Asimismo, el tránsito obligatorio por servidores de retransmisión añade una sobrecarga de latencia que no puede mitigarse con enlaces directos punto a punto (P2P).

7. Mobizen

Desarrollada por la empresa surcoreana RSUPPORT Co., Ltd., Mobizen introdujo un cambio de paradigma en el acceso remoto al extender el control bidireccional hacia plataformas móviles heterogéneas (Android e iOS) desde estaciones de trabajo de escritorio y visores basados en navegador web. A diferencia de las herramientas orientadas a servidores y computadoras personales, el control remoto de dispositivos móviles enfrenta restricciones severas impuestas por el sistema operativo anfitrión: limitaciones de energía térmica y batería, aislamiento estricto de procesos mediante cajas de arena (*sandboxing*) y la ausencia histórica de controladores gráficos estándar para la captura de pantalla e inyección de eventos táctiles (RSUPPORT Co., Ltd., 2024).

7.1. Arquitectura de réplica y control para dispositivos móviles

La arquitectura de Mobizen implementa un modelo cliente-servidor invertido respecto a la jerarquía de hardware convencional: el dispositivo móvil opera como servidor anfitrión que procesa y transmite la interfaz visual, mientras que la computadora personal actúa como cliente o visor interactivo.



Imagen 9: Arquitectura de réplica móvil, captura por MediaProjection e inyección de eventos en Mobizen (RSUPPORT Co., Ltd., 2024).

Como ilustra la Img. 9, la arquitectura móvil de Mobizen articula tres subsistemas funcionales:

1. **Agente móvil anfitrión:** Aplicación que reside en el dispositivo móvil y coordina la captura del búfer de cuadro, la codificación de video en tiempo real y la recepción de eventos periféricos sintéticos.
2. **Motor de codificación acelerado (RSCC / H.264):** Para minimizar la sobrecarga en la CPU móvil y mitigar la degradación por estrangulamiento térmico (*thermal throttling*), RSUPPORT desarrolló el códec RSCC (*RSUPPORT Screen Capture Codec*), complementado con la canalización nativa de compresión por hardware H.264/AVC a través de la interfaz `MediaCodec` de Android. Esto permite generar flujos de video con compresión adaptativa optimizada para la nitidez de texto y gráficos de interfaz de usuario.
3. **Consola y visor remoto:** Aplicación nativa para Windows/macOS o visor web basado en WebRTC/HTML5, encargado de decodificar el flujo audiovisual, escalar la resolución gráfica y capturar eventos de ratón y teclado para traducirlos a gestos táctiles.

7.2. Modos de conexión: enlace USB (ADB) y transporte inalámbrico

Para adaptarse a diversos escenarios operativos y restricciones de infraestructura, Mobizen soporta dos esquemas de enlace físico y de red:

- **Enlace físico mediante USB (ADB):** Aprovecha el protocolo del puente de depuración de Android (ADB, *Android Debug Bridge*). Al conectar el dispositivo mediante cable USB y habilitar la depuración USB, el cliente de escritorio ejecuta el comando `adbforwardtcp:PORTtcp:PORT`, mapeando un puerto TCP local de la computadora con el conector del agente en el dispositivo móvil. Este enlace por hardware elimina la fluctuación de retardo (jitter) y las pérdidas de paquetes propias de las redes inalámbricas, alcanzando tasas de refresco de hasta 60 fotogramas por segundo con latencias inferiores a 15 milisegundos.
- **Transporte inalámbrico (Wi-Fi y redes móviles):** En entornos locales, el visor y

el dispositivo móvil descubren sus direcciones mediante protocolos de difusión local (mDNS/UPnP) y establecen un enlace directo TCP/UDP. Si el dispositivo se conecta mediante redes celulares (3G, LTE o 5G) o se ubica detrás de cortafuegos simétricos, la sesión se enruta a través de servidores de retransmisión gestionados por RSUPPORT en la nube. La seguridad del emparejamiento se valida mediante un desafío de autenticación mutua de dos pasos basado en códigos numéricos dinámicos de seis dígitos, códigos QR o tokens de sesión vinculados a la cuenta del usuario.

7.3. Servicios del sistema operativo móvil, permisos de accesibilidad y captura

El mayor obstáculo en la ingeniería de control remoto móvil radica en las políticas de seguridad del núcleo de Android, diseñadas bajo el principio de aislamiento estricto entre aplicaciones:

- **Evolución de la captura en el subsistema de pantalla:** En versiones tempranas de Android (anteriores a Android 5.0), el sistema operativo carecía de una API pública para leer el búfer de cuadro. Mobizen solventaba esta limitación mediante el *Modo Booster*, ejecutando binarios auxiliares con privilegios de depuración vía ADB para leer de `/dev/graphics/fb0` o invocar la utilidad interna `screencap`. A partir de Android 5.0 (API 21), el software migró al marco oficial `MediaProjectionManager` (Google Developers, 2024b). Dicho subsistema despliega un diálogo de sistema donde el usuario autoriza la captura; tras otorgarse, el kernel expide un token que instancia una pantalla virtual (`VirtualDisplay`) enlazada a una superficie de `MediaCodec`, extrayendo fotogramas por GPU sin requerir privilegios de superusuario (root).
- **Inyección de eventos sintéticos y accesibilidad:** El modelo de seguridad de Android prohíbe que una aplicación ordinaria inyecte pulsaciones sobre la ventana de otra aplicación, reservando el permiso privilegiado `INJECT_EVENTS` exclusivamente a firmas del sistema operativo. Históricamente, RSUPPORT superó esto mediante acuerdos comerciales con fabricantes OEM para distribuir motores auxiliares firmados con claves de sistema (*Mobizen for SAMSUNG* o *Mobizen for LG*) o ejecutando comandos mediante el demonio `adbd` en modo USB (`inputtap` e `inputswipe`). En versiones modernas de Android (API 24 o superior), la plataforma aprovecha la API de servicios de accesibilidad (`AccessibilityService`) (Google Developers, 2024a). Tras concederse el permiso en los ajustes del sistema, el agente invoca el método nativo `dispatchGesture()`, construyendo trayectorias sintéticas (`GestureDescription`) que el despachador de ventanas procesa de forma nativa e indistinguible de los toques físicos.

8. Análisis comparativo de herramientas de escritorio remoto

El análisis individual de las soluciones examinadas (Chrome Remote Desktop, LogMeIn, TeamViewer, ShowMyPC y Mobizen) evidencia que, si bien todas persiguen proyectar el entorno interactivo en red, sus decisiones arquitectónicas difieren sensiblemente. La selección de la pila de transporte, los mecanismos de enlace con el núcleo del sistema operativo y los esquemas de penetración de cortafuegos reflejan compromisos deliberados de ingeniería entre fidelidad visual, latencia, seguridad criptográfica y portabilidad operativa.

8.1. Matriz de capacidades técnicas, protocolos y rendimiento

La Tabla 2 sintetiza las dimensiones estructurales y operativas de las tecnologías analizadas.

Tabla 2: Matriz comparativa de tecnologías de escritorio remoto analizadas.

Herramienta	Pila y códec gráfico	Transporte y topología	Integración en el SO	Seguridad y cifrado	Modelo de despliegue
Chrome Remote Desktop	WebRTC (RFC 8825); video VP8, VP9, H.264 por GPU	UDP preferente; ICE, STUN y TURN (443). Enlace P2P directo o relay	DDA en Windows; PipeWire DMA-BUF y `Xvfb` en Linux; Mojo IPC	DTLS/SRTP (AES-128/256); autenticación de PIN vía SPAKE2	Cliente web PWA sin instalación + agente anfitrión
LogMeIn (GoTo)	Protocolo gráfico propietario con compresión en servidor	Túnel TCP saliente HTTPS (443) hacia clúster pasarela; P2P o relay	Servicio del sistema; captura DDA; suplantación SSPI (`LogonUser`)	TLS 1.2/1.3 con AES-256 (CBC/GCM) de extremo a extremo	MSI corporativo (`DeployToken`) para GPO/MECM y portal web
TeamViewer	Protocolo binario propietario con compresión adaptativa	TCP/UDP híbrido (5938) con repliegue a 443 y 80; P2P o router relay	Servicio con inicio automático; DDA; retardo exponencial ante fallos	RSA-4096 (PFS) + AES-256 simétrico de extremo a extremo	Cliente completo, Host desatendido y QuickSupport portable
ShowMyPC	RFB (RFC 6143) sobre TightVNC; soporte opcional de RDP	Túnel inverso SSHv2 saliente hacia servidor intermedio público	TightVNC en bucle local (`127.0.0.1`); inyección en Win32k	Encapsulación SSHv2 (AES + HMAC); clave efímera de 12 dígitos	Binario monolítico autocontenido sin instalación previa
Mobizen	Códec RSCC y compresión H.264 acelerada vía `MediaCodec`	Enlace USB físico por ADB (`adbforward`) o Wi-Fi/celular con relay	Pantalla virtual vía `MediaProjection`; inyección `dispatchGesture`	Canales TLS + verificación en dos pasos (código PIN o QR)	Agente móvil (Android/iOS) + consola de escritorio o navegador

A partir de la taxonomía expuesta, se observan diferencias críticas en el diseño del transporte. Las herramientas que priorizan la inmediatez interactiva y la fluidez en tasas de refresco elevadas (Chrome Remote Desktop y TeamViewer) optan por protocolos no orientados a la conexión fundamentados en datagramas UDP. Como demuestran Alvestrand (2021) y TeamViewer Germany GmbH (2024b), la transmisión de cuadros comprimidos mediante UDP desacoplada de los canales de datos SCTP/DTLS elimina el fenómeno de bloqueo de cabeza de línea (*head-of-line blocking*), donde la pérdida transitoria de un paquete de red no interrumpe el procesamiento de los cuadros gráficos subsiguientes. En contraposición, las arquitecturas sustentadas estrictamente sobre flujos orientados a la conexión mediante TCP (LogMeIn y ShowMyPC) garantizan entrega ordenada incondicional pero exhiben degradación de latencia ante pérdidas de paquetes en enlaces congestionados o inalámbricos.

8.2. Evaluación de modelos de despliegue, seguridad y limitaciones de plataforma

El balance entre accesibilidad y robustez de seguridad determina el campo de aplicación de cada solución:

- **Soberanía criptográfica:** TeamViewer y Chrome Remote Desktop destacan por implementar arquitecturas de conocimiento cero (*zero-knowledge architecture*) en la capa de datos. En TeamViewer, el intercambio asimétrico RSA de 4096 bits negocia claves simétricas efímeras que residen exclusivamente en los nodos terminales, impidiendo que los servidores maestros o de enrutamiento descifren los paquetes en

tránsito (TeamViewer Germany GmbH, 2024b). De forma análoga, Chrome Remote Desktop recurre al protocolo PAKE/SPAKE2 para autenticar el PIN local sin transmitirlo a los servidores de Google (The Chromium Projects, 2024). Por el contrario, en esquemas como ShowMyPC y pasarelas de retransmisión convencionales, la seguridad descansa en la confianza depositada en la infraestructura del proveedor que termina o conmuta los túneles SSH.

- **Aislamiento de privilegios y elevación del sistema:** En plataformas de escritorio tradicionales, el agente anfitrión requiere privilegios elevados (`LocalSystem` en Windows o servicios raíz en UNIX) para interceptar sesiones multiusuario, capturar pantallas de bloqueo e interactuar con secuencias de atención segura (`Ctrl+Alt+Supr`). LogMeIn destaca en este ámbito al subordinar el acceso remoto a la identidad local del sistema operativo mediante validación nativa en SSPI (GoTo Technologies USA, Inc., 2024). En contraste, entornos móviles como Android (Mobizen) imponen una arquitectura de aislamiento rígido donde los servicios de fondo carecen de acceso irrestricto al subsistema gráfico, obligando al usuario a renovar periódicamente el consentimiento explícito mediante `MediaProjectionManager` (Google Developers, 2024b) y habilitar manualmente permisos en el marco de accesibilidad (Google Developers, 2024a).
- **Gestión de flotas frente a soporte puntual:** Mientras herramientas como TeamViewer QuickSupport y ShowMyPC maximizan la agilidad en sesiones de asistencia técnica reactiva al no exigir privilegios de instalación ni modificaciones en el registro de la máquina anfitriona, soluciones empresariales como LogMeIn Central y las directivas de Chrome Remote Desktop permiten gobernar centralizadamente parques computacionales masivos mediante directivas de grupo (GPO), control estricto de listas blancas de dominios y distribución silenciosa de tareas por lotes.

9. Pruebas de funcionamiento práctico con TeamViewer

El análisis experimental de una plataforma de escritorio remoto exige contrastar los modelos teóricos de transporte e intercepción gráfica frente a las contingencias operativas de redes heterogéneas y subsistemas de entrada y salida dispares. En esta sección se evalúa el comportamiento funcional de TeamViewer en dos escenarios representativos: la interacción simétrica entre computadoras de escritorio y portátiles, y el control interactivo multiplataforma con dispositivos móviles.

9.1. Entorno experimental, topología de red y metodología de instrumentación

Para aislar las variables que inciden en la latencia, el ancho de banda y el consumo de recursos de procesamiento, se configuró un banco de pruebas compuesto por tres nodos computacionales heterogéneos:

- **Nodo anfitrión principal (PC de escritorio):** Estación de trabajo con procesador x86_64 multinúcleo, 32 GB de memoria RAM, adaptador gráfico acelerado por hardware y sistema operativo Windows 11 Enterprise (versión 23H2). La máquina se conectó al conmutador central mediante un enlace físico Ethernet Gigabit (IEEE 802.3ab) con una resolución nativa de 1920×1080 píxeles a una frecuencia de refresco de 60 Hz.

- **Nodo cliente portátil (Laptop):** Computadora portátil con procesador x86_64, 16 GB de memoria RAM y distribución Linux Ubuntu 24.04 LTS (núcleo Linux 6.8 con servidor de despliegue Wayland). La conectividad se estableció mediante una tarjeta de red inalámbrica Wi-Fi 6 (IEEE 802.11ax) sobre una banda de 5 GHz con modulación dinámica de canal.
- **Dispositivo móvil (Cliente/Host alterno):** Teléfono inteligente con procesador móvil de arquitectura ARM64, 8 GB de memoria RAM y sistema operativo Android 14. Dispone de conectividad dual conmutada entre enlace Wi-Fi local (802.11ac) y red celular de banda ancha 4G LTE / 5G NSA.

La evaluación se instrumentó bajo dos topologías de red diferenciadas: un escenario de red de área local (*local area network*, LAN) intradominio, donde ambos nodos compartían el mismo prefijo de enrutamiento privado (`192.168.1.0/24`) bajo un conmutador de baja latencia; y un escenario de red de área amplia (*wide area network*, WAN) transdominio, en el cual el nodo cliente operó a través de un proveedor de servicios externo con enrutamiento NAT de nivel operador (*carrier-grade* NAT, CGNAT), forzando el descubrimiento de candidatos y el atravesamiento de pasarelas públicas de TeamViewer (TeamViewer Germany GmbH, 2024b).

La telemetría del sistema se midió de forma continua registrando el tiempo de ida y vuelta (*round-trip time*, RTT) de los paquetes de control y video, la tasa efectiva de transferencia en red (kbps y Mbps), el porcentaje de ocupación de la unidad central de procesamiento (CPU) atribuido al proceso de TeamViewer, y la cadencia de entrega de fotogramas por segundo (*frames per second*, FPS).

9.2. Opción 1: Control entre computadoras de escritorio y portátiles

En la primera modalidad de evaluación se estableció una sesión interactiva bidireccional desde la computadora portátil cliente hacia la estación de trabajo anfitriona. El procedimiento inició con el intercambio del identificador numérico persistente de 9 dígitos y la autenticación mediante contraseña efímera generada por el agente anfitrión, verificada de manera cifrada a través de los servidores maestros de TeamViewer (TeamViewer Germany GmbH, 2024c). Tras la negociación criptográfica inicial, los nodos establecieron un túnel UDP directo mediante técnicas de perforación de puertos, prescindiendo de la intermediación de pasarelas de retransmisión durante el intercambio masivo de fotogramas.

El comportamiento de los subsistemas del sistema operativo durante la sesión remota evidenció las siguientes características técnicas (Img. 10):

- **Dinámica de puntero y latencia interactiva:** En el entorno LAN, el RTT promedio se mantuvo por debajo de los 12 ms, permitiendo habilitar la renderización del cursor en el extremo cliente con retroalimentación inmediata. En el entorno WAN, con un RTT medio de 48 ms, el protocolo activó la predicción posicional local, mitigando la sensación de retraso al desplazar ventanas o seleccionar texto en editores de código y suites ofimáticas.
- **Sincronización del portapapeles:** El agente anfitrión se integró con la API del portapapeles del sistema operativo (`CF_UNICODETEXT` y formatos MIME de mapa de bits). Las operaciones de copia en el cliente transmitieron asíncronamente el contenido hacia el host mediante un canal de control auxiliar multiplexado sobre el flujo de sesión, posibilitando la transferencia instantánea de texto plano, código fuente y fragmentos binarios sin degradar la tasa de fotogramas del canal gráfico.



Imagen 10: Telemetría operativa y métricas de rendimiento en sesión remota PC a Laptop (TeamViewer Germany GmbH, 2024c).

- **Transferencia bidireccional de archivos:** Mediante el subsistema dedicado de navegación del sistema de archivos, se realizaron pruebas de envío y recepción de paquetes comprimidos de 500 MB. En enlace LAN directo, la velocidad media sostenida alcanzó los 72 MB/s, operando cerca de los límites del canal inalámbrico Wi-Fi 6, con verificación estricta de sumas de comprobación criptográficas SHA-256 por cada bloque recibido antes de su confirmación en disco.
- **Inyección de secuencia de atención segura (SAS):** La invocación remota de la combinación de teclas `Ctrl+Alt+Supr` fue procesada satisfactoriamente por el servicio en segundo plano de TeamViewer (`TeamViewer_Service.exe`), el cual, al ejecutarse bajo la cuenta de privilegios máximos `LocalSystem`, invocó directamente la función `SendSAS()` del subsistema Winlogon de Windows, permitiendo desbloquear la estación, iniciar sesión desatendida y acceder al Administrador de Tareas sin requerir interacción física local.

9.3. Opción 2: Control interactivo con dispositivos móviles

La incorporación de dispositivos móviles al esquema de administración remota introduce restricciones severas asociadas a la ausencia de periféricos físicos convencionales (ratón y teclado mecánico) y al aislamiento riguroso que los sistemas operativos móviles imponen sobre sus procesos de usuario. Para validar esta interoperabilidad, se ejecutaron dos procedimientos complementarios:

9.3.1. Control de estación de escritorio desde dispositivo móvil (Móvil a PC)

Utilizando la aplicación TeamViewer Remote Control en el dispositivo móvil Android sobre conectividad celular 4G LTE, se accedió a la estación de trabajo anfitriona. La interfaz implementa un esquema de traducción de eventos táctiles hacia primitivas de puntero del sistema operativo de escritorio:

- Un toque simple sobre la pantalla táctil capacitiva se traduce en un evento de clic izquierdo (`WM_LBUTTONDOWN` y `WM_LBUTTONUP` en Windows).
- La pulsación prolongada (*long press*) emula el clic derecho mediante el mensaje `WM_RBUTTONDOWN`, desplegando los menús contextuales en el entorno remoto.
- El deslizamiento simultáneo con dos dedos se mapea directamente a los mensajes de la rueda del ratón (`WM_MOUSEWHEEL`), facilitando el desplazamiento vertical y horizontal en documentos extensos.
- El gesto de pellizco (*pinch-to-zoom*) ajusta dinámicamente la escala del búfer de cuadro virtual en la memoria del cliente móvil sin alterar la resolución nativa de pantalla del nodo anfitrión.

Para la entrada de caracteres alfanuméricos, la aplicación superpone un teclado virtual optimizado que incorpora una barra persistente con modificadores del sistema operativo ausentes en teclados móviles convencionales, tales como `Ctrl`, `Alt`, `Shift`, la tecla del sistema operativo (`Super` o tecla Windows), `Esc` y el bloque de funciones de `F1` a `F12`. Esto permite ejecutar combinaciones avanzadas de administración de consola y comandos de cierre de aplicaciones remotas.



Imagen 11: Control interactivo y flujo de inyección de eventos en sesiones con dispositivos móviles (TeamViewer Germany GmbH, 2024a).

9.3.2. Control remoto del dispositivo móvil desde estación de escritorio (PC a Móvil)

En el procedimiento inverso, se instaló en el dispositivo móvil la herramienta TeamViewer QuickSupport complementada con el módulo universal de control (TeamViewer Germany GmbH, 2024a). Este escenario representa un desafío de ingeniería superior en el sistema operativo Android, debido a que el marco de seguridad prohíbe que una aplicación ordinaria acceda al búfer de cuadro global o inyecte eventos de entrada en el espacio de otras aplicaciones (Img. 11).

Para sortear estas restricciones sin vulnerar el núcleo ni requerir privilegios de superusuario (root), el agente móvil solicita al usuario la concesión de un token en tiempo de ejecución a través de la API `MediaProjectionManager` (Google Developers, 2024b). Una vez otorgado el permiso, el servicio de captura obtiene una superficie virtual (`VirtualDisplay`)

ligada a un codificador por hardware H.264/AVC, el cual serializa el flujo de pantalla en paquetes UDP hacia la estación de escritorio cliente.

Simultáneamente, la inyección de eventos periféricos se canaliza mediante el servicio de accesibilidad del sistema operativo (`AccessibilityService`). Cuando el operador del cliente de escritorio realiza un clic o arrastre con el ratón en la ventana de réplica, las coordenadas normalizadas son transmitidas hacia el dispositivo móvil, donde el servicio invoca la función nativa `dispatchGesture()` (Google Developers, 2024a). Esta rutina sintetiza trazos táctiles continuos con precisión milimétrica, permitiendo interactuar con menús de configuración del teléfono, abrir aplicaciones instaladas y emular pulsaciones en los botones de navegación del sistema operativo móvil (`Atrás`, `Inicio` y `Aplicaciones recientes`).

10. Conclusiones personales

El estudio sistemático y la evaluación experimental de las tecnologías de escritorio remoto permiten comprender que estas plataformas no constituyen simples aplicaciones de usuario montadas sobre la red, sino extensiones arquitectónicas complejas que se acoplan íntimamente con los subsistemas más profundos del sistema operativo. La evolución histórica desde los primeros emuladores de terminal basados en flujos de caracteres hasta las modernas canalizaciones de video aceleradas por hardware refleja un continuo esfuerzo de la ingeniería por trasladar la experiencia de interactividad local a través de medios de transmisión intrínsecamente hostiles y variables. Lejos de limitarse a la captura ingenua de mapas de bits completos, la viabilidad del control remoto contemporáneo descansa en la capacidad del núcleo y de las interfaces de programación gráfica para aislar únicamente las regiones rectangulares modificadas, delegando la compresión de cuadros en motores de codificación dedicados en la GPU y manteniendo colas de sincronización asíncronas que desacoplan el refresco visual de la latencia impuesta por la infraestructura de telecomunicaciones.

Uno de los aprendizajes técnicos más determinantes extraídos del análisis comparativo radica en la tensión inherente entre la confiabilidad estricta del transporte y la preservación del tiempo real interactivo. La tradicional dependencia de los protocolos orientados a la conexión sustentados sobre flujos TCP, si bien garantiza una reconstrucción íntegra y ordenada de los paquetes, introduce una penalización inaceptable en escenarios de red congestionados debido al fenómeno de bloqueo de cabeza de línea. Cuando un datagrama que transporta un fotograma se extravía o se retarda, la insistencia del protocolo por retransmitirlo antes de liberar los datos subsiguientes paraliza el flujo interactivo, acumulando un retraso perceptivo que destruye la respuesta del sistema. En contraposición, la consolidación de WebRTC y de arquitecturas híbridas no orientadas a la conexión fundamentadas en datagramas UDP demuestra que, en el plano del control gráfico interactivo, la frescura y la vigencia temporal de los cuadros superan a la completitud histórica de los datos, permitiendo descartar paquetes obsoletos para mantener la sincronía del puntero y la fidelidad del cursor sin degradar la sesión.

Asimismo, la investigación pone de manifiesto la radical divergencia arquitectónica que existe entre los esquemas de privilegios de los sistemas operativos de escritorio convencionales y los modelos modernos de aislamiento en entornos móviles. En sistemas como Windows y distribuciones UNIX tradicionales, las herramientas de acceso remoto operan históricamente mediante servicios privilegiados del sistema que gozan de facultades irrestrictas para interceptar escritorios seguros, capturar pantallas de autenticación institucional

e inyectar secuencias de atención crítica reservadas a la interacción física local. Por el contrario, la arquitectura de espacio aislado implementada en sistemas operativos móviles modernos demuestra un paradigma contemporáneo de seguridad por diseño, donde ningún proceso de usuario puede acceder al búfer de cuadro global ni alterar los eventos periféricos de aplicaciones adyacentes sin la mediación expresa de marcos de consentimiento en tiempo de ejecución. Esta barrera impone desafíos de ingeniería notables para la administración desatendida, pero establece una contención indispensable frente a vectores de ataque basados en la captura silenciosa de información confidencial.

En el ámbito de la seguridad y la soberanía de la información, el atravesamiento de barreras de traducción de direcciones de red y cortafuegos mediante pasarelas intermedias suscita reflexiones fundamentales sobre la confianza en infraestructuras compartidas. Si bien las pasarelas centralizadas resuelven de forma transparente la indisponibilidad de direcciones IPv4 públicas, introducen un punto de concentración que podría comprometer la privacidad si no se acompaña de una criptografía robusta. Las plataformas que implementan esquemas de conocimiento cero mediante el intercambio asimétrico efímero de claves públicas en los nodos terminales demuestran que es posible utilizar servidores de retransmisión públicos sin facultar a sus operadores para descifrar el contenido del búfer de cuadro ni las pulsaciones de teclado. Esta rigurosa separación entre el plano de enrutamiento y el plano de confidencialidad de datos constituye un principio rector de diseño que debe gobernar cualquier solución tecnológica distribuida en entornos empresariales e industriales modernos.

Finalmente, el desarrollo integral de este análisis consolida una visión profesional crítica en ingeniería en computación sobre la convergencia interdisciplinaria requerida para construir software de infraestructura. Comprender a detalle la interacción entre el planificador de procesos del sistema operativo, la gestión de memoria virtual, la aceleración gráfica en hardware, los protocolos estandarizados de atravesamiento de NAT como STUN, TURN e ICE, y los algoritmos criptográficos modernos, transforma la percepción que se tiene sobre el funcionamiento de las redes y los sistemas operativos. El escritorio remoto deja de ser una utilidad cotidiana para convertirse en un objeto de estudio riguroso que evidencia cómo la adecuada orquestación de llamadas al sistema, estructuras de datos en memoria y protocolos de comunicaciones hace posible superar las distancias físicas, garantizando la continuidad operativa y la administración eficiente de recursos computacionales con altos estándares de rendimiento y seguridad.

Referencias

Alvestrand, H. (2021). *Overview: Real-Time Protocols for Browser-Based Applications* (inf. téc. N.º RFC 8825). IETF. Consultado el 2 de noviembre de 2026, desde https://www.rfc-editor.org/rfc/rfc8825

Anka, M. (2007). *LogMeIn: An In-Depth Look* (inf. téc.). LogMeIn, Inc. Consultado el 2 de noviembre de 2026, desde https://www.goto.com

Carretero Pérez, J., García Carballeira, F., Miguel Anasagasti, P. d., & Pérez Costoya, F. (2001). *Sistemas operativos: una visión aplicada* (1.ª ed.). McGraw-Hill/Interamericana de España.

Deitel, H. M., Deitel, P. J., & Choffnes, D. R. (2004). *Operating systems* (3.ª ed.). Pearson Prentice Hall.

Google Chrome Enterprise. (2024). *Chrome enterprise policy list: Remote desktop policies*. Consultado el 2 de noviembre de 2026, desde https://chromeenterprise.google/policies/

Google Developers. (2024a). *Accessibilityservice and gesture dispatching in Android*. Consultado el 2 de noviembre de 2026, desde https://developer.android.com/reference/android/accessibilityservice/AccessibilityService

Google Developers. (2024b). *Mediaprojection and virtualdisplay API reference*. Consultado el 2 de noviembre de 2026, desde https://developer.android.com/reference/android/media/projection/MediaProjection

Google Help. (2024). *Access another computer with chrome remote desktop*. Consultado el 2 de noviembre de 2026, desde https://support.google.com/chrome/answer/1649523

GoTo Technologies USA, Inc. (2024). *LogMeIn Pro and Central: Security and architecture overview*. Consultado el 2 de noviembre de 2026, desde https://www.goto.com/security

Kaplinsky, C. (2001). *TightVNC: Fast and free remote control utility*. Consultado el 2 de noviembre de 2026, desde https://www.tightvnc.com/

Microsoft Corporation. (2024). *[MS-RDPBCGR]: Remote desktop protocol: Basic connectivity and graphics remoting*. Consultado el 2 de noviembre de 2026, desde https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-rdpbcgr/5073f4ed-1e93-45e1-b0e9-96e353f4c664

Petit-Huguenin, M., Salgueiro, G., Rosenberg, J., Wing, D., & Mahy, R. (2020). *Session Traversal Utilities for NAT (STUN)* (inf. téc. N.º RFC 8489). IETF. Consultado el 2 de noviembre de 2026, desde https://www.rfc-editor.org/rfc/rfc8489

Postel, J., & Reynolds, J. (1983). *Telnet Protocol Specification* (inf. téc. N.º RFC 854). IETF. Consultado el 2 de noviembre de 2026, desde https://www.rfc-editor.org/rfc/rfc854

Reddy, T., Wing, D., Mahy, R., & Uberti, J. (2020). *Traversal Using Relays around NAT (TURN): Relay Extensions to Session Traversal Utilities for NAT (STUN)* (inf. téc. N.º RFC 8656). IETF. Consultado el 2 de noviembre de 2026, desde https://www.rfc-editor.org/rfc/rfc8656

Richardson, T., & Levine, P. (2011). *The Remote Framebuffer Protocol* (inf. téc. N.º RFC 6143). IETF. Consultado el 2 de noviembre de 2026, desde https://www.rfc-editor.org/rfc/rfc6143

Richardson, T., Stafford-Fraser, Q., Wood, K. R., & Hopper, A. (1998). Virtual Network Computing. *IEEE Internet Computing*, *2*(1), 33-38. https://doi.org/10.1109/4236.656088

Rosenberg, J., Keranen, A., Lowekamp, B. B., & Roach, A. (2018). *Interactive Connectivity Establishment (ICE): A Protocol for Network Address Translator (NAT) Traversal* (inf. téc. N.º RFC 8445). IETF. Consultado el 2 de noviembre de 2026, desde https://www.rfc-editor.org/rfc/rfc8445

RSUPPORT Co., Ltd. (2024). *Mobizen: Real-time mobile screen mirroring and remote control architecture*. Consultado el 2 de noviembre de 2026, desde https://www.mobizen.com/

Scheifler, R. W., & Gettys, J. (1986). The X Window System. *ACM Transactions on Graphics*, *5*(2), 79-109. https://doi.org/10.1145/22949.24053

Shockey, J. (2008). *LogMeIn: An In-Depth Look at Remote Access Security* (inf. téc.). SANS Institute. Consultado el 2 de noviembre de 2026, desde https://www.sans.org/white-papers/

ShowMyPC LLC. (2024). *ShowMyPC documentation: Remote support, screen sharing, and architecture*. Consultado el 2 de noviembre de 2026, desde https://www.showmypc.com/

Silberschatz, A., Galvin, P. B., & Gagne, G. (2006). *Fundamentos de sistemas operativos* (7.ª ed.). McGraw-Hill/Interamericana de España.

Srisuresh, P., & Egevang, K. (2001). *Traditional IP Network Address Translator (Traditional NAT)* (inf. téc. N.º RFC 3022). IETF. Consultado el 2 de noviembre de 2026, desde https://www.rfc-editor.org/rfc/rfc3022

Stallings, W. (2018). *Operating systems: Internals and design principles* (9.ª ed.) [Global Edition]. Pearson Education Limited.

TeamViewer Germany GmbH. (2024a). *TeamViewer mobile remote control and quicksupport administrator guide*. Consultado el 2 de noviembre de 2026, desde https://www.teamviewer.com/en/documents/

TeamViewer Germany GmbH. (2024b). *TeamViewer Security Whitepaper: Core Principles, Architecture, and Technical Specifications* (inf. téc.). TeamViewer Germany GmbH. Consultado el 2 de noviembre de 2026, desde https://www.teamviewer.com/security/

TeamViewer Germany GmbH. (2024c). *TeamViewer user guide: Remote control and unattended access*. Consultado el 2 de noviembre de 2026, desde https://community.teamviewer.com/

The Chromium Projects. (2024). *Chromoting: Chrome remote desktop architecture and design*. Consultado el 2 de noviembre de 2026, desde https://chromium.googlesource.com/chromium/src/+/main/remoting/README.md

Ylonen, T., & Lonvick, C. (2006). *The Secure Shell (SSH) Protocol Architecture* (inf. téc. N.º RFC 4251). IETF. Consultado el 2 de noviembre de 2026, desde https://www.rfc-editor.org/rfc/rfc4251